Auto

I WANT TO CREATE A CUSTOM MADE BTC MINING WEB APP , THAT IS USER FRIENDLY AND IS SIMPLE TO USE BUT ALSO HAS ALOT OF METRICS TO ENSURE THE SOFTWARE IS MINING PROERPLY. I WILL BE USING AN M2 MAC MINI . (DONT FOCUS ON COMPUTING POWER RIGHT NOW)

I WANT TO USE HTML5 AND PHP AND EITHER API OR WEB-SOCKET TO CONNECT TO BTC CORE NETWORK (WHICHEVER IS FASTER) . ADD ANYTHING YOU THINK THE MINER WOULD NEED

CREATE A TXT FILE ON WHAT NEEDS TO BE DONE, ILL BE USING CURSOR

Thought for 16s

Below is the content of the TXT file you requested, outlining a step-by-step plan to build your custom BTC mining web app. I've named it "btc_mining_web_app_plan.txt" for reference. This plan assumes you're building a web-based interface that interacts with Bitcoin Core (running locally on your M2 Mac Mini) to handle mining operations. Note: Actual BTC mining on an M2 Mac Mini will be extremely inefficient due to hardware limitations (CPU-only, no ASIC support), but as per your instructions, I'm not focusing on computing power. Instead, the focus is on creating a user-friendly web app with metrics monitoring.

The app will use:

- HTML5 for the frontend (simple, responsive UI).
- PHP for the backend (to handle RPC calls to Bitcoin Core).
- Bitcoin Core's RPC API for connecting to the BTC network (it's HTTP-based and standard; WebSockets aren't natively supported by Bitcoin Core, so RPC is faster and more direct for core interactions. However, I'll incorporate WebSockets via PHP for real-time frontend updates on metrics to make it feel responsive).
- Additional elements: I'll add security features (e.g., authentication), logging, error handling, and extensive metrics (hash rate, network stats, wallet balance, etc.). Also adding pool support (since solo mining is impractical), user config options, and alerts for issues like high CPU usage or network disconnects.

Copy-paste this into a file named "btc_mining_web_app_plan.txt" in your Cursor project directory, and use it as a roadmap.

---

btc_mining_web_app_plan.txt

Project Overview:

- Goal: Build a web app that serves as a [illegible]rd for starting/stopping BTC mining,

Quick Answer

Ask anything

- Architecture:
  - Frontend: HTML5 + CSS + JavaScript (for UI and WebSocket client for real-time updates).
  - Backend: PHP (handles Bitcoin Core RPC calls, processes metrics, and serves data via API endpoints or WebSockets).
  - Database: Optional SQLite for storing logs and historical metrics (simple, no external DB needed).
  - Connection: Bitcoin Core RPC (HTTP/JSON-RPC) for core mining/network ops – faster than WebSockets for batched commands. Use Ratchet (PHP WebSocket library) for pushing real-time metrics to the frontend.
  - Extras Added: Support for mining pools (e.g., via Stratum protocol proxy in PHP), user authentication, email/SMS alerts for critical events (e.g., via Twilio API), auto-backup of wallet, and visualization charts (using Chart.js).
- Prerequisites:
  - Install Bitcoin Core on your M2 Mac Mini (download from bitcoin.org; enable RPC in bitcoin.conf with rpcuser, rpcpassword, and rpcport=8332).
  - Set up a local web server (e.g., via MAMP or built-in PHP server on macOS).
  - Install Composer for PHP dependencies.
  - Ensure your Mac Mini has Node.js if you want to use npm for frontend builds (optional).

Step-by-Step Development Plan:

1. Set Up Project Structure:
   - Create a new directory in Cursor: e.g., /btc-miner-app/
   - Subdirs:
     - /public/ (for frontend: index.html, styles.css, app.js)
     - /src/ (for PHP backend: classes for RPC client, metrics processor)
     - /vendor/ (for Composer dependencies)
     - /logs/ (for storing app logs and metrics history)
     - /config/ (for config.php with RPC credentials, pool settings)
   - Initialize Composer: Run `composer init` in the root dir.
   - Install PHP deps:
     - composer require cboden/ratchet (for WebSockets)
     - composer require mike42/escpos-php (optional, if you want printer support for logs)
     - composer require monolog/monolog (for logging)
     - composer require twilio/sdk (for alerts, if adding SMS)
     - composer require predis/predis (if caching metrics for performance)
2. Configure Bitcoin Core Integration:
   - In /config/bitcoin.conf (for Bitcoin Core): Add rpcallowip=127.0.0.1, rpcuser=youruser, rpcpassword=yourpass, server=1, txindex=1 (for full transaction indexing if needed for metrics).
   - Create a PHP RPC Client class (/src/BitcoinRPC.php):

- Use curl or Guzzle to make JSON-RPC calls to http://127.0.0.1:8332/.
- Methods: getblocktemplate() for mining templates, submitblock() for submitting solved blocks, getmininginfo() for metrics, getnetworkinfo(), getwalletinfo(), etc.
- Handle authentication with base64-encoded user:pass.
- Add error handling: Catch RPC errors (e.g., connection refused) and log them.

- Add Pool Support (essential for realistic mining):
  - Use a PHP Stratum client library (e.g., composer require slush0/stratum-mining-proxy, but adapt for client mode).
  - Allow user to input pool URL (e.g., stratum+tcp://pool.example.com:3333), username, password in UI.
  - Fallback to solo mining via Bitcoin Core if no pool.

3. Build Backend PHP Logic:
   - Create /src/MinerController.php:
     - Start/Stop Mining: Use a background PHP process (e.g., via proc_open) to run a mining loop that fetches templates, computes hashes (using PHP's hash functions for SHA256, but note: this is CPU mining – inefficient).
     - Metrics Collection: Poll Bitcoin Core every 10-60s for:
       - Hash rate (local and network)
       - Difficulty
       - Block height (local vs. network)
       - Shares submitted/accepted/rejected (if pooled)
       - Wallet balance and recent transactions
       - CPU usage (use sys_getloadavg() or exec('top') on macOS)
       - Network connections/peers
       - Errors/warnings (e.g., orphan blocks)
       - Custom: Uptime, estimated earnings (based on hash rate and BTC price via free API like coingecko.com).
     - Real-Time Push: Use Ratchet to set up a WebSocket server (/src/websocket_server.php). Run it as a daemon (php websocket_server.php). Push metrics updates to connected clients.
     - API Endpoints: Use simple PHP routing (or Slim framework) for /api/start, /api/stop, /api/config, /api/metrics (JSON responses).
     - Security: Implement basic auth (sessions or JWT), sanitize inputs, use HTTPS if deploying.
     - Logging: Use Monolog to log all actions/errors to /logs/miner.log.
     - Alerts: If hash rate drops or errors occur, send email (mail()) or SMS (Twilio).
4. Build Frontend (HTML5 + JS):
   - index.html: Simple layout with sections:
     - Dashboard: Start/Stop buttons, config form (pool URL, wallet address).

- Metrics Panel: Real-time displays (use with IDs for updates).
  - Gauges/Charts: Integrate Chart.js for hash rate graph, pie chart for share acceptance.
  - Tables: For recent blocks, transactions, peers.
  - Alerts: Popup for issues (e.g., "Network disconnected!").
- Responsive: Use CSS media queries for mobile/desktop.

- app.js:
  - WebSocket Client: Connect to ws://localhost:8080 (your Ratchet server) for live metrics updates. On message, update DOM elements.
  - API Calls: Use fetch() to call PHP endpoints (e.g., fetch('/api/start')).
  - User-Friendly Features: Tooltips on metrics, simple onboarding wizard, dark mode toggle.
  - Validation: JS checks for valid inputs (e.g., wallet address format).

5. Add Essential Miner Features:
   - Auto-Restart: If mining crashes, auto-restart via cron job or PHP loop.
   - Wallet Management: Buttons to generate new addresses, backup wallet.dat.
   - Performance Optimizations: Cache metrics in Redis, throttle polls to avoid overloading.
   - Security Extras: Rate limiting on API, CSRF protection, encrypt sensitive config.
   - Testing Tools: Add a simulation mode (fake metrics) for dev without real mining.
   - Internationalization: Basic support for multiple languages if needed.
   - Backup/Restore: Auto-backup config and logs.
6. Testing and Deployment:
   - Local Test: Run Bitcoin Core, start WebSocket server, open http://localhost in browser.
   - Unit Tests: Use PHPUnit for PHP classes (e.g., test RPC calls).
   - Edge Cases: Test with no network, invalid pool, high load.
   - Deployment: On Mac Mini, use nginx/Apache for production. Set up as a service with systemd/supervisor.
   - Monitoring: Add self-monitoring (e.g., app checks if Bitcoin Core is running).
7. Potential Enhancements:
   - Integrate external APIs: BTC price from CoinGecko, weather effects on energy (joke, but could add power usage estimate).
   - Mobile App Bridge: PWA support for notifications.
   - Community Features: Share metrics anonymously via a public API.

Timeline Estimate (assuming part-time dev):

- Week 1: Setup + Backend RPC.
- Week 2: Metrics + WebSockets.
- Week 3: Frontend + Testing.
- Total: 3-4 weeks.

Resources:

- Bitcoin Core RPC Docs: https://developer.bitcoin.org/reference/rpc/
- Ratchet WebSockets: https://github.com/ratchetphp/Ratchet
- Chart.js: https://www.chartjs.org/
- If stuck, search StackOverflow or Bitcoin StackExchange.

This plan is comprehensive but modular – start with core RPC integration. If you need code snippets or refinements, ask!

Bitcoin Core RPC examples

Mining pool comparisons

more detailed code snippets